

RAP 後端開發練習 3：Behavior Definition 基礎——Managed 與 Unmanaged 對照

Lecture

為什麼這一課要教兩種寫法

rap01 介紹過 Managed / Unmanaged 的概念差異，也提過本課程只教 Managed (Unmanaged 需要紮實的 OOP 底子，份量太大)。但開發過程中意外發現一個對這系統很關鍵的事實：這套系統的 RAP **Managed Runtime (CUD 寫入部分)** 被 SAP 官方標記為「尚未對外釋出」，任何 Managed BDEF 的 **create/update/delete** 操作，執行到底層都會直接 Dump (詳見下方 Part C)。所以這一課調整成：

- **Part A：Managed 語法**——當作知識儲備，跟官方教材 / 未來的 ABAP Cloud RAP 課程接軌，語法在這系統上能寫、能啟用，但沒辦法真正執行
- **Part B：Unmanaged 語法**——這系統上真正能跑、能驗證的版本，份量比 rap01 原本規劃的重一些，但既然 Managed 執行不了，這是這系統唯一能讓你看到「資料真的寫進去了」的路
- **Part C：兩者的差異總表 + 為什麼這系統只有 Unmanaged 能跑**

Part A：Managed BDEF 語法 (知識儲備，這系統無法執行)

Managed 的意思是：CRUD 的實際資料庫存取 (**INSERT/UPDATE/DELETE**) 由框架自動產生，你只需要用宣告式語法告訴框架「這個實體支援哪些操作」：

```
managed;

define behavior for ZI_RAP02_TASK alias Task
persistent table zrap02_task
etag created_at
lock master
{
    create;
    update;
    delete;

    field ( mandatory ) description;
    field ( readonly ) created_at, created_by;
}
```

逐段拆解：

- **managed;**：header 只有這一個字，宣告 Managed 模式。本系統不支援 **strict(2)** 這種新式語法 (rap01 已提過)。
- **define behavior for ZI_RAP02_TASK alias Task**：綁定到 rap02 建好的 CDS View，**alias Task** 幫這個實體取一個簡短的別名，EML 呼叫時用這個別名。

- **persistent table zrap02_task**：告訴框架資料實際存在哪張表——因為 CDS View 欄位名稱跟表格欄位名稱完全一致，這裡不需要額外的 **mapping for** 區塊。
- **lock master**：這個實體是鎖定管理的「主體」，框架自動處理 **ENQUEUE/DEQUEUE**。
- **etag created_at**：ETag (樂觀鎖定用的版本標記) 指定用 **created_at** 這個欄位。
- **{ create; update; delete; }**：宣告這個實體支援哪些標準操作。
- **field (mandatory) description : description** 是必填欄位，**create** 時沒填會被框架擋下來。
- **field (readonly) created_at, created_by**：這兩個欄位使用者不能直接透過 **create/update** 設值——為 rap05 (Determination) 鋪路。

⚠ 這一課踩到的語法坑：**etag master** 在這系統不支援。官方教材 / 新式 ABAP Cloud RAP 常見寫法是 **etag master created_at** (帶 **master** 關鍵字)，這系統啟用直接報 **"authorization | draft | late | { | ~" expected, not "created_at"**。——錯誤訊息列出的合法後續 **token** 完全沒有 **etag**，代表這系統文法不認得 **etag master** 這個組合。回頭比對 rap01 提過的系統既有標準物件 (**SCR_E_DBDEV / A_ProductionSupplyArea**)，兩者都是寫單純 **etag <欄位>**，完全沒有 **master**——這是這系統適用的語法 (見 **.claude/rules/sap-adt-mcp.md** 第 40.11 節)。

Managed 版本的 EML 語法 (**ZR_RAP03_DEMO**，見答案區) 也已經寫好、語法正確、成功啟用，可以當範例讀，但**不要期待它能真正執行成功**——原因見 Part C。

Part B：Unmanaged BDEF——這系統上真正能跑的版本

Unmanaged 的意思是：CRUD 的實際邏輯**你自己寫在一個 ABAP 類別裡**，框架只負責派發呼叫，不管你怎麼存資料。這一課用一個簡化的獨立範例示範 (**ZRAP03_UMTEST / ZI_RAP03_UMTEST**，只有 **id / descr** 兩個欄位，刻意比 rap02 的 **Task** 簡單，聚焦在 Managed vs Unmanaged 的機制差異，不重複設計資料模型)：

BDEF (注意跟 Managed 版本的語法差異)：

```
implementation unmanaged in class zbp_i_rap03_um4 unique;

define behavior for ZI_RAP03_UMTEST alias Test
lock master
{
    create;
}
```

跟 Part A 的 Managed BDEF 對照，兩個關鍵差異：

- **header 是 implementation unmanaged in class <類別名> unique;**，不是單純 **managed;**——因為所有邏輯都要靠這個類別實作，BDEF 必須指名是哪個類別
- **完全沒有 persistent table 子句**——Managed 需要這個子句告訴框架「資料存在哪張表，讓框架自動生成 SQL」；Unmanaged 不需要，因為資料庫存取全部由你自己在實作類別裡寫，框架不插手

實作類別：Global 類別本體只是空殼，真正的 Handler 邏輯寫在類別的 **Local Types Include**：

```
" Global 類別本體 ( ZBP_I_RAP03_UM4 )
CLASS zbp_i_rap03_um4 DEFINITION
PUBLIC
ABSTRACT
```

```

FINAL
FOR BEHAVIOR OF zi_rap03_umtest.

PUBLIC SECTION.
PROTECTED SECTION.
PRIVATE SECTION.

ENDCLASS.

CLASS zbp_i_rap03_um4 IMPLEMENTATION.
ENDCLASS.

```

```

" Local Types Include ( 真正的 Handler 邏輯 )
CLASS lcl_handler DEFINITION INHERITING FROM cl_abap_behavior_handler.
PRIVATE SECTION.
    METHODS lock FOR LOCK
        IMPORTING it_lock FOR LOCK test.

    METHODS create FOR MODIFY
        IMPORTING it_create FOR CREATE test.

    METHODS read FOR READ
        IMPORTING it_read FOR READ test RESULT et_result.
ENDCLASS.

CLASS lcl_handler IMPLEMENTATION.

METHOD lock.
ENDMETHOD.

METHOD create.
    LOOP AT it_create INTO DATA(ls_create).
        INSERT zrap03_umtest FROM @( VALUE #( client = sy-mandt id = ls_create-id descr
= ls_create-descr ) ).
    ENDLOOP.
ENDMETHOD.

METHOD read.
    LOOP AT it_read INTO DATA(ls_key).
        SELECT SINGLE id, descr FROM zrap03_umtest WHERE id = @ls_key-id INTO
@DATA(ls_data).
        IF sy-subrc = 0.
            APPEND VALUE #( %key = ls_key-%key id = ls_data-id descr = ls_data-descr ) TO
et_result.
        ENDIF.
    ENDLOOP.
ENDMETHOD.

```

```
ENDLOOP.  
ENDMETHOD.  
  
ENDCLASS.
```

這段寫法是照抄這系統既有標準物件 `CL_SD_BEHV_SALESORDERMANAGE` (`C_SalesOrderManage` 的實作類別，rap01 提過的標準 `Unmanaged` 範例) 讀出來的真實模式，不是憑空猜的。幾個重點：

- **INHERITING FROM `cl_abap_behavior_handler`**：所有 `Unmanaged Handler` 類別的必要基底類別，提供 `failed/reported/mapped` 這些保護屬性給你在方法裡直接用 (不用自己宣告)
- **FOR LOCK / FOR MODIFY / FOR READ**：方法用途分類，`IMPORTING ... FOR CREATE <alias>` 這種寫法把方法綁定到 BDEF 宣告的特定操作
- **LOCK 方法必須實作** (就算是空的) ——即使沒有真的要做什麼鎖定邏輯，框架也要求這個方法存在
- **READ 方法也建議實作**——即使 BDEF 的 `{}` 裡沒有明寫 `read`，`Unmanaged` 實體通常還是需要基礎的讀取能力 (`EML / OData` 內部會用到)

⚠️ 這一課實測踩到的兩個坑：

1. `VALUE #( mandt = ... )` 寫錯——這張表的 `Client` 欄位自己取名叫 `client` (不是內建保留字 `mandt`)，`VALUE` 建構子要對應欄位實際名稱，寫 `mandt =` 會啟用失敗，這只是單純欄位名稱對應錯誤，不是 RAP 語法限制。
2. `READ` 方法結果表的技术鍵欄位是 `%key`，不是 `%tky`——啟用時系統直接在錯誤訊息裡建議了正確名稱 (`No component exists with the name "%TKY", but there is a component with a similar name "%KEY"`)。

✅ 驗證結果 (`programrun` 無頭執行，完全成功)：

```
before EML  
after EML, failed is initial: X  
after commit entities  
DB check OK, descr = Unmanaged Test
```

資料真的寫進資料庫了，而且是無頭執行成功的——這點也意外推翻了先前記錄的「`EML` 沒辦法用 `programrun` 無頭驗證」(見 Part C 的完整解釋)。

Part C：Managed vs Unmanaged 差異總表，以及為什麼這系統只有 `Unmanaged` 能跑

	Managed	Unmanaged
CRUD 邏輯寫在哪	框架自動生成，你只宣告 <code>create/update/delete</code>	你自己寫在實作類別的 <code>LOCK/CREATE/UPDATE/DELETE/READ</code> 方法裡
BDEF header	<code>managed;</code>	<code>implementation unmanaged in class <類別> unique;</code>
需不需要 <code>persistent table</code>	需要	不需要 (資料庫存取自己處理)

	Managed	Unmanaged
需不需要額外的實作類別	不需要 (除非要加 Determination / Validation / Action)	一定需要
程式碼份量	少	多 (含 OOP、SQL 語句都要自己寫)
這系統能不能真正執行 CUD	✗ (見下方原因)	✓ 已驗證成功
適合的情境	全新設計的資料物件 (官方主推方向)	包一層 RAP 介面在既有系統/邏輯之上 (這系統的標準物件 <code>C_SalesOrderManage</code> 就是這樣)

為什麼這系統的 **Managed CUD** 一律失敗：使用者在 SAP GUI 執行 Managed BDEF 的 EML CREATE，得到 Runtime Error (`MESSAGE_TYPE_X_TEXT`，元件 `BC-ESI-RAP-CSP`)，根本原因是 `CL_CSP_MD_METADATA_FACTORY` 裡一段檢查邏輯——這個 CDS Entity 所在的套件，必須在一份硬編碼白名單裡 (`SBOI_RAP_CSP_TST%` 套件前綴、或幾個 SAP 內部套件/元件)，不在清單裡 (我們的 `$TMP` 或任何自訂套件都不在) 就用致命訊息擋下來，程式碼裡甚至留了開發者自己寫的英文註解：「csp isn't released for public usage until now」。查證 SAP Community 找到一篇 **2019-11** 一字不差的同款回報，時間點跟這套系統的 **S/4HANA 1909** 高度吻合——這是 **1909 版本 Managed Runtime** 尚未對客戶開放寫入功能的已知限制，不是我們哪裡設定錯了。這個檢查只卡在真正寫入的時候，純讀取 (如 Fiori Elements Preview 顯示清單) 不受影響。

意外收穫：這也讓我們搞清楚了先前的一個誤判——`.claude/rules/sap-adt-mcp.md` 第 42 節原本記錄「EML 沒辦法用 `programrun` 無頭驗證」，但這次用 Unmanaged BDEF 的 EML 測試完全無頭執行成功，證實問題從來不是 **EML 這個語言機制**，而是呼叫到 Managed Runtime 時觸發的致命 Dump，透過 RFC Bridge 傳回時卡住了連線。這個推測已經更正。

學習目標

- 能寫出這系統適用的 Managed BDEF 語法 (`managed;`、`persistent table`、`lock master`、`etag <欄位>` 不含 `master`)，知道這是知識儲備、銜接未來 ABAP Cloud RAP 課程用
- 能寫出這系統適用的 Unmanaged BDEF 語法 (`implementation unmanaged in class ... unique;`，無 `persistent table`)，知道跟 Managed 的語法差異
- 能寫出 Unmanaged 實作類別的基本結構：繼承 `cl_abap_behavior_handler`、`FOR LOCK/FOR MODIFY/FOR READ` 方法綁定語法
- 能講出 Managed / Unmanaged 的完整差異對照 (程式碼份量、適用情境、這系統的可執行性)
- 知道這系統 Managed Runtime CUD 無法執行的具體原因 (`CL_CSP_MD_METADATA_FACTORY` 白名單機制，SAP 官方尚未對客戶套件開放，推測跟 S/4HANA 1909 版本有關)
- 能寫出基礎的 EML 語法 (`MODIFY ENTITIES OF ... CREATE`、`COMMIT ENTITIES`)，知道 `CREATE/UPDATE` 用 `WITH VALUE`、`DELETE` 用 `FROM VALUE`

物件清單

物件	名稱	型別	可執行性
Managed Behavior Definition	<code>ZI_RAP02_TASK</code> (沿用 rap02 的 CDS View)	BDEF/BDO	語法正確，CUD 無法執行

物件	名稱	型別	可執行性
Managed EML 驗證程式	ZR_RAP03_DEMO	PROG/P	語法正確，執行會 Dump
Unmanaged 測試表格	ZRAP03_UMTEST	TABL/DT	—
Unmanaged CDS View	ZI_RAP03_UMTEST	DDLS/DF	—
Unmanaged Behavior Definition	ZI_RAP03_UMTEST	BDEF/BDO	✓
Unmanaged 實作類別	ZBP_I_RAP03_UM4	CLAS/OC	✓
Unmanaged EML 驗證程式	ZR_RAP03_UMTEST	PROG/P	✓ 已驗證成功

全部物件都在 \$TMP 套件，sap_inactive_objects 確認 0 筆殘留。

驗證方式

1. **Managed** 部分：checkruns API / sap_inactive_objects 確認語法正確、成功啟用即可——**不要嘗試執行 ZR_RAP03_DEMO**，不管在 SAP GUI 或 programrun，一律會遇到 Part C 說明的 Dump，這是預期中的、已經查清楚原因的結果，不用重複驗證
2. **Unmanaged** 部分：已用 programrun 完整驗證成功，輸出見上方 Part B，資料庫比對也確認寫入成功

思考題

1. field (readonly) created_at, created_by (Managed 版本) 現在阻止了使用者自己填這兩個欄位，但也沒有任何機制會自動填值——rap05 要怎麼補上「自動填值」這個邏輯？（提示：想想 Determination 的 on save/on modify 時機；但別忘了 Managed CUD 這系統執行不了，這一題純粹是語法層面的思考）
2. Unmanaged 版本的 create 方法，如果同時要寫入兩張表（例如 Header+Item），實作邏輯要怎麼設計？跟 Managed 版本只要在 BDEF 宣告 Composition 就能自動處理，複雜度差在哪裡？
3. 如果之後真的解除了這系統 Managed Runtime 的白名單限制（例如 Support Package 升級後），Part A 寫的 Managed BDEF 語法要改多少才能真正動起來？（提示：語法本身可能完全不用改，只是「能不能執行」這件事跟語法無關）

答案

Managed：zi_rap02_task.bdef.abap、zr_rap03_demo.prog.abap。 **Unmanaged**：zrap03_umtest.tabl.abap、zi_rap03_umtest.ddls.abap、zi_rap03_umtest.bdef.abap、zbp_i_rap03_um4.clas.abap（Global 類別本體）、zbp_i_rap03_um4.clas.locals_imp.abap（Local Handler 實作）、zr_rap03_umtest.prog.abap（EML 驗證程式，已驗證執行成功）。